Santiago Cuenu

Problema 2 — Sistema de reservas de vuelos

Una start-up de viajes desarrolla una plataforma para reservas de vuelos internacionales.

Requisitos

- Dos destinos
- Tres clases de asientos (econo, normal, premium)
- Gestión de reservas
- Disponibilidad de asientos

DISEÑO

1. ANÁLISIS DEL PROBLEMA

Entradas

- nombre(str): Nombre completo del usuario
- apellido(str): apellido usuario
- ID_usuario: int
- destino: str
- asiento(str):Selección entre Económica, Normal o Premium
- ID_asiento(int): numero de silla

Salidas:

- Mensaje de confirmación de reserva exitosa o error si el asiento está ocupado.
- Tiquete o resumen de la reserva con el precio final calculado según la clase elegida.
- Lista de asientos disponibles y ocupados por destino

Restricciones:

- El sistema solo operará con dos destinos internacionales fijos
- Reserva Económica Paga únicamente el precio base del vuelo
- Reserva Normal Paga el precio base más un recargo del 15% por equipaje delicado.
- *Reserva Premium* Paga el precio base más un recargo del 40% por acceso VIP.
- Un pasajero no puede reservar un número de asiento que ya se encuentre ocupado

Reglas de negocio

1. Solo se permite la reserva de vuelos que existan y correspondan al destino seleccionado.
2. Un asiento únicamente puede ser reservado si se encuentra disponible.
3. Un asiento ocupado no puede ser asignado nuevamente mientras no sea liberado.
4. Cada reserva debe estar asociada a un identificador único de usuario.
5. Los datos del pasajero deben ser válidos antes de registrar la reserva.
6. El precio final de la reserva depende de la clase seleccionada (Económica, Normal o Premium).

7. La clase de la reserva se determina según el rango del asiento seleccionado.
8. El número de asiento debe estar dentro del rango permitido por la capacidad del vuelo.
9. Al eliminar una reserva, el asiento debe volver a estar disponible.
10. No se permite eliminar reservas inexistentes o que no correspondan al usuario.
11. El sistema debe garantizar la persistencia de los datos (vuelos, usuarios y reservas).
12. Las consultas deben mostrar únicamente la información asociada al usuario correspondiente.

CLASES

Clase: vuelo

Atributos privados:

- __destino: str
- __capacidad_maxima: int
- __asientos_disponibles: list
- __avion: int
- __precio_vuelo: float
- __asientos_disponibles: list
- __fecha: str

Métodos públicos:

- get_destino(): str
- get_capacidad_maxima(): int
- get_asientos_disponibles(): list
- get_avion(): int
- get_precio_vuelo(): float
- ocupar_asiento(ID_asiento: int): bool (Revisa si el asiento está en la lista de disponibles)
- __init__(destino: str, capacidad_maxima: int, num_avion: int, precio_vuelo: float)
- mostrar_informacion(): none

SuperClase: Reserva

Atributos privados:

- __nombre_pasajero : str
- __apellido_usuario:str
- __destino : str
- __ID_asiento : int
- __precio_base : float
- __id_usuario: int
- __ID_avion:int

Métodos públicos:

- __init__(nombre_pasajero: str, apellido_usuario: str, id_usuario: int, destino: str, ID_asiento: int, precio_base: float, id_avion:int)
- get_nombre_pasajero(): str
- get_apellido_usuario(): str
- get_id_usuario(): int
- get_destino(): str
- get_ID_asiento(): int
- get_precio_base(): float
- calcular_precio_final(): float
- mostrar_detalles(): str
- get_ID_Avion(): int

SubClase: Economica

Atributos privados: (Heredados de la superclase)

- __nombre_pasajero : str
- __apellido_usuario:str
- __destino : str
- __ID_asiento : int
- __precio_base : float
- __id_usuario: int

Metodos publicos:

- __init__(nombre_pasajero: str, apellido_usuario: str, id_usuario: int, destino: str, ID_asiento: int, precio_base: float, id_avion:int): Llama al init de la superclase.
- calcular_precio_final(): float

SubClase: Normal

Atributos privados: (Heredados de la superclase)

- __nombre_pasajero : str
- __apellido_usuario:str
- __destino : str
- __ID_asiento : int
- __precio_base : float
- __id_usuario: int

Metodos publicos:

- __init__(nombre_pasajero: str, apellido_usuario: str, id_usuario: int, destino: str, ID_asiento: int, precio_base: float, id_avion:int): Llama al constructor de la superclase.
- calcular_precio_final(): float

SubClase: Premium

Atributos privados: (Heredados de la superclase)

- __nombre_pasajero : str
- __apellido_usuario:str
- __destino : str
- __ID_asiento : int
- __precio_base : float
- __id_usuario: int

Metodos publicos:

- __init__(nombre_pasajero: str, apellido_usuario: str, id_usuario: int, destino: str, ID_asiento: int, precio_base: float, id_avion:int): Llama al constructor de la superclase.
- calcular_precio_final(): float

Clase: Usuario

Atributos privados:

- __nombre_pasajero : str
- __id_usuario: int
- __apellido_usuario: str

Métodos públicos:

- __init__(nombre_pasajero: str, apellido_usuario: str, id_usuario: int)
- get_nombre_pasajero(): str
- get_apellido_usuario(): str
- get_id_usuario(): int

Clase: PantallaAsientos

Atributos privados:

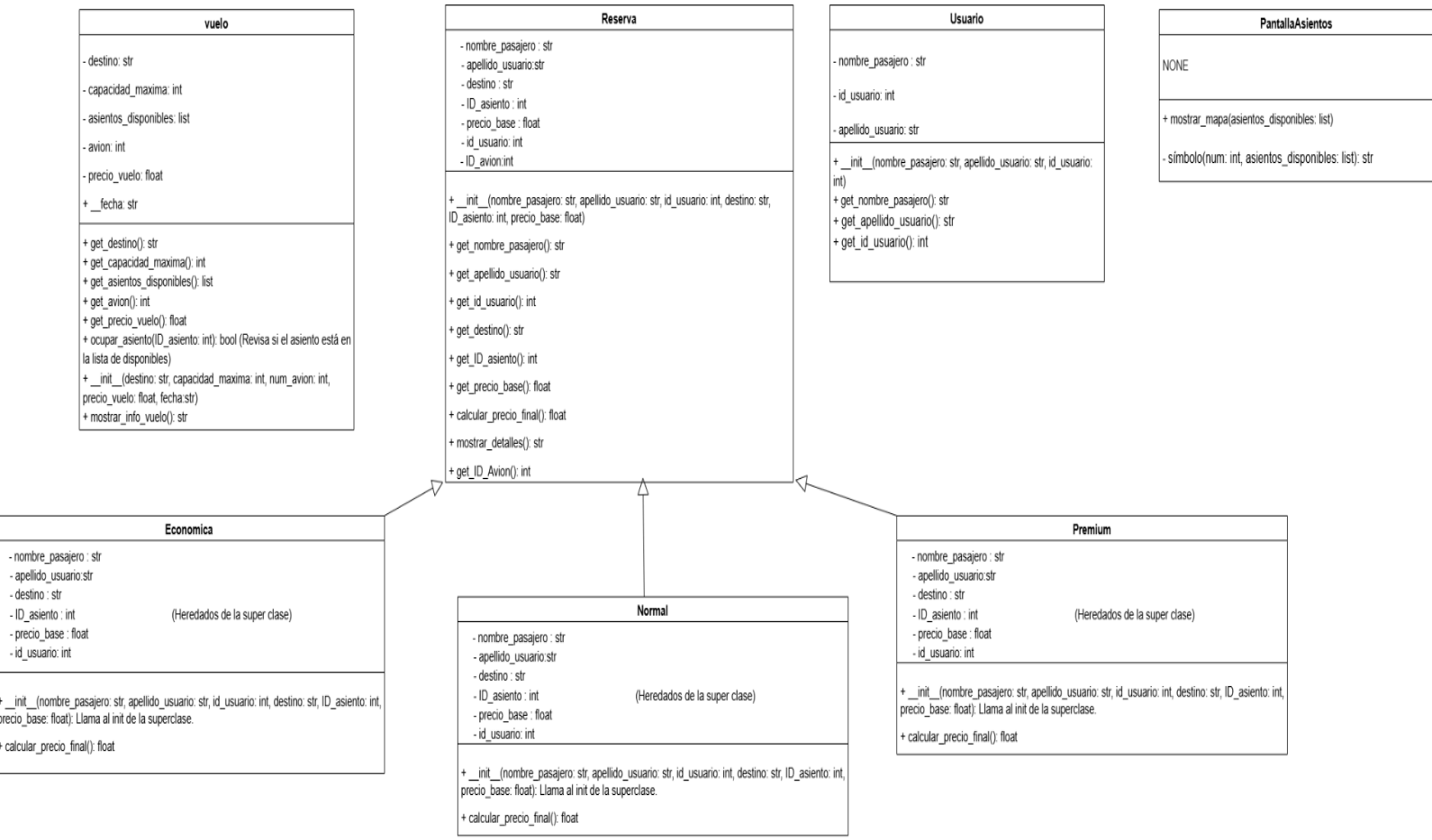
- none

Métodos públicos:

- mostrar_mapa(asientos_disponibles: list)

Métodos privados:

- - símbolo(num: int, asientos_disponibles: list): str



2. Diagrama de clases

3. Pseudcodigos

- **Clase: Vuelo**

__destino
__capacidad_maxima
__avion
__precio_vuelo
__asientos_disponible
__fecha

Método __init__(destino: str, capacidad: int, num_avion: int, precio: float, fecha: str)
Inicio
__destino = destino
__capacidad_maxima = capacidad
__avion = num_avion
__precio_vuelo = precio
__fecha = fecha
__asientos_disponibles = {Conjunto del 1 hasta capacidad}

```
Fin

Método ocupar_asiento(ID_asiento: int) : bool
Inicio
   IF ID_asiento ESTÁ EN __asientos_disponibles:
      Remover ID_asiento de __asientos_disponibles
      Retornar Verdadero
   ELSE:
      Retornar Falso
   FIN IF
Fin

Método get_destino() : str
Inicio
   Retornar __destino
Fin

Método get_avion() : int
Inicio
   Retornar __avion
Fin

Método get_precio_vuelo() : float
Inicio
   Retornar __precio_vuelo
Fin

Método get_asientos_disponibles() : set
Inicio
   Retornar __asientos_disponibles
Fin

Método mostrar_info_vuelo() :
Inicio
   Retornar Vuelo  __avion   a  __destino
         " | Fecha:  __fecha
         " | Precio: $  __precio_vuelo
         " | Asientos disponibles:  +tamaño(__asientos_disponibles)
Fin

FIN CLASE
```

- **SuperClase: Reserva**

```
__nombre_pasajero : str
__apellido_usuario : str
__id_usuario : int
__destino : str
__ID_asiento : int
```

__precio_base : float
__ID_avion:int

Método __init__(nombre, apellido, id, destino, silla, precio)
Inicio
    __nombre_pasajero = nombre
    __apellido_usuario = apellido
    __id_usuario = id
    __destino = destino
    __ID_asiento = silla
    __precio_base = precio
Fin

Método calcular_precio_final() : float
Inicio
    Retornar __precio_base
Fin

Método get_precio_base() : float
Inicio
    Retornar __precio_base
Fin

- **SubClase: Económica**

Método __init__(nombre, apellido, id, destino, silla, precio)
Inicio
    Llamar a Super.__init__(nombre, apellido, id, destino, silla, precio)
Fin

Método calcular_precio_final() : float
Inicio
    Retornar get_precio_base()
Fin

- **SubClase: Normal**

Método __init__(nombre, apellido, id, destino, silla, precio)
Inicio
    Llamar a Super.__init__(nombre, apellido, id, destino, silla, precio)
Fin

Método calcular_precio_final() : float
Inicio
    precio = get_precio_base()
    Retornar precio + (precio * 0.15)
Fin

- **SubClase: Premium**

```
Método __init__(nombre, apellido, id, destino, silla, precio)
Inicio
   Llamar a Super.__init__(nombre, apellido, id, destino, silla, precio)
Fin

Método calcular_precio_final() : float
Inicio
   precio = get_precio_base()
   Retornar precio + (precio * 0.40)
Fin
```

- **Clase: Usuario**

```
__nombre_pasajero : str
__apellido_usuario : str
__id_usuario : int



Método __init__(nombre, apellido, id)
Inicio
   __nombre_pasajero = nombre
   __apellido_usuario = apellido
   __id_usuario = id
Fin

Método get_nombre_pasajero() : str
Inicio
   Retornar __nombre_pasajero
Fin

Método get_apellido_usuario() : str
Inicio
   Retornar __apellido_usuario
Fin

Método get_id_usuario() : int
Inicio
   Retornar __id_usuario
Fin
```

- **Clase: PantallaAsiento**

```
Método __simbolo(num: int, asientos_disponibles: list) : str
Inicio
   IF num ESTÁ EN asientos_disponibles:
      Retornar "🟢" + Texto(num)
   ELSE:
      Retornar "🔴" + Texto(num)
```

```
    FIN IF
Fin

Método mostrar_mapa(asientos_disponibles: list)
Inicio
    Imprimir "--- MAPA DE ASIENTOS ---"
    FOR cada num desde 1 hasta 35:
        icono = __simbolo(num, asientos_disponibles)
        Imprimir icono
    FIN FOR
Fin

cias por usar la aerolínea"

                    SALIR DEL PROGRAMA



        FIN SEGÚN



        FIN MIENTRAS



FIN SI



FIN FUNCION
```

- **Aplicación principal**

```
FUNCION hacer_reserva(vue, usu, res)



MIENTRAS VERDADERO HACER



        MOSTRAR "hola bienvenido a la aerolínea"

        MOSTRAR "seleccione su destino"

        MOSTRAR "1. Madrid"

        MOSTRAR "2. Paris"
```

```
MOSTRAR "3. Roma"
MOSTRAR "4. Londres"

LEER opcion

SI opcion = "1" ENTONCES
PARA cada i EN vue HACER
SI i.destino = "Madrid" ENTONCES
        MOSTRAR i.info_vuelo
FIN SI
FIN PARA

SINO SI opcion = "2" ENTONCES
PARA cada i EN vue HACER
SI i.destino = "Paris" ENTONCES
        MOSTRAR i.info_vuelo
FIN SI
FIN PARA

SINO SI opcion = "3" ENTONCES
PARA cada i EN vue HACER
SI i.destino = "Roma" ENTONCES
        MOSTRAR i.info_vuelo
FIN SI
FIN PARA
```

```
SINO SI opcion = "4" ENTONCES
PARA cada i EN vue HACER
SI i.destino = "Londres" ENTONCES
        MOSTRAR i.info_vuelo
FIN SI
FIN PARA

SINO
MOSTRAR "Opción no válida. Por favor, seleccione una opción válida."
SALIR
FIN SI

MOSTRAR "por favor ingrese el codigo de su vuelo a reservar"
LEER cod_avion

vuelo_seleccionado = NONE

PARA cada i EN vue HACER
SI i.codigo = cod_avion ENTONCES
vuelo_seleccionado = i
SALIR DEL PARA
FIN SI
FIN PARA

SI vuelo_seleccionado = NULO ENTONCES
MOSTRAR "Codigo no encontrado. Por favor, ingrese un codigo válido."
```

```
CONTINUAR
FIN SI

MOSTRAR mapa_asientos(vuelo_seleccionado)

MOSTRAR "ingrese el numero de asiento que desea reservar"
LEER asiento

SI asiento <= 10 O asiento >= 1 ENTONCES

SI i.ocupar_asiento(asiento) ENTONCES

LEER persona
LEER ape

MOSTRAR "Asiento reservado con éxito."

id_usu = número aleatorio entre 1000 y 9999

CREAR nuevo_usuario(persona, ape, id_usu)
AGREGAR nuevo_usuario A usu

CREAR reserva_premium(persona, ape, id_usu, i.destino, cod_avion, asiento,
i.precio)
AGREGAR reserva A res

MOSTRAR detalles_premium
```

```
MOSTRAR mapa_asientos(i)

SALIR

SINO
MOSTRAR "Asiento no disponible. Por favor, elija otro asiento."
FIN SI

SINO SI asiento <= 20 O asiento >= 11 ENTONCES

SI i.ocupar_asiento(asiento) ENTONCES

LEER persona
LEER ape

MOSTRAR "Asiento reservado con éxito."

id_usu = número aleatorio

CREAR nuevo_usuario
AGREGAR a usu

CREAR reserva_normal
AGREGAR a res
```

MOSTRAR detalles_normal

SINO

MOSTRAR "Asiento no disponible. Por favor, elija otro asiento."

FIN SI

SINO SI asiento > 20 O asiento <= 35 ENTONCES

SI i.ocupar_asiento(asiento) ENTONCES

LEER persona

LEER ape

MOSTRAR "Asiento reservado con éxito."

id_usu = número aleatorio

CREAR nuevo_usuario

AGREGAR a usu

CREAR reserva_economica

AGREGAR a res

MOSTRAR detalles_economica

SINO

```
        MOSTRAR "Asiento no disponible. Por favor, elija otro asiento."
        FIN SI

        FIN SI

FIN MIENTRAS

FIN FUNCION
PROCEDIMIENTO hacer_reserva(vue, usu, res)

MIENTRAS VERDADERO HACER

        MOSTRAR "--- RESERVA DE VUELO ---"
        MOSTRAR "1. Madrid"
        MOSTRAR "2. Paris"
        MOSTRAR "3. Roma"
        MOSTRAR "4. Londres"

        LEER opcion

        destinos = {
        "1":"Madrid",
        "2":"Paris",
        "3":"Roma",
        "4":"Londres"
        }
```

```
SI opcion NO ESTÁ EN destinos ENTONCES
MOSTRAR "Opción inválida"
CONTINUAR
FIN SI

destino = destinos[opcion]

MOSTRAR "Vuelos disponibles"

PARA cada v EN vue HACER
SI v.destino = destino ENTONCES
MOSTRAR v.info_vuelo
FIN SI
FIN PARA

MIENTRAS VERDADERO HACER

LEER cod_avion

SI longitud(cod_avion) >= 4 Y
primeras 2 letras SON letras Y
resto SON números ENTONCES
SALIR DEL MIENTRAS
SINO
MOSTRAR "Código inválido"
```

```
FIN SI

FIN MIENTRAS

vuelo = None

PARA cada v EN vue HACER
SI v.codigo = cod_avion Y v.destino = destino ENTONCES
vuelo = v
SALIR DEL PARA
FIN SI
FIN PARA

SI vuelo = NONE ENTONCES
MOSTRAR "Vuelo no encontrado"
CONTINUAR
FIN SI

MOSTRAR mapa_asientos(vuelo)

MIENTRAS VERDADERO HACER

INTENTAR
LEER asiento

SI 1 <= asiento <= 35 ENTONCES
```

```
        SALIR DEL MIENTRAS
SINO
        MOSTRAR "Fuera de rango"
FIN SI

CAPTURAR ERROR
MOSTRAR "Debe ingresar un número"
FIN INTENTAR

FIN MIENTRAS

SI NO vuelo.ocupar_asiento(asiento) ENTONCES
MOSTRAR "Asiento ocupado"
CONTINUAR
FIN SI

MIENTRAS VERDADERO HACER
LEER nombre
SI nombre contiene solo letras ENTONCES
SALIR
SINO
MOSTRAR "Solo letras"
FIN SI
FIN MIENTRAS

MIENTRAS VERDADERO HACER
```

LEER apellido

SI apellido contiene solo letras ENTONCES

SALIR

SINO

MOSTRAR "Solo letras"

FIN SI

FIN MIENTRAS

id_usu = número aleatorio entre 1000 y 9999

CREAR usuario(nombre, apellido, id_usu)

AGREGAR usuario A usu

SI 1 <= asiento <= 10 ENTONCES

CREAR reserva_premium(nombre, apellido, id_usu, destino, cod_avion, asiento, precio_vuelo)

SINO SI 11 <= asiento <= 20 ENTONCES

CREAR reserva_normal(nombre, apellido, id_usu, destino, cod_avion, asiento, precio_vuelo)

SINO

CREAR reserva_economica(nombre, apellido, id_usu, destino, cod_avion, asiento, precio_vuelo)

FIN SI

AGREGAR reserva A res

```
        MOSTRAR "Reserva exitosa"
        MOSTRAR detalles_reserva

        SALIR

FIN MIENTRAS

FIN FUNCION
FUNCION eli_reserva(reser, vuelos)

MOSTRAR "Ingrese su ID de usuario para revisar sus reservas:"
LEER id_usuario

reservas_usuario = [ ]
PARA cada i EN reser HACER
        SI id_usuario = i.id_usuario ENTONCES
        MOSTRAR i.detalles
        AGREGAR i A reservas_usuario
        FIN SI
FIN PARA

SI reservas_usuario está vacía ENTONCES
        MOSTRAR "No tienes reservas registradas."
        SALIR
FIN SI
```

```
MOSTRAR "Ingrese el ID del vuelo para eliminar la reserva:"
LEER id_vuelo

eliminado = eliminar_reserva(reser, vuelos, id_usuario, id_vuelo)

SI eliminado ENTONCES
    MOSTRAR "Reserva eliminada con éxito"
SINO
    MOSTRAR "No se encontró la reserva"
FIN SI


MOSTRAR "--- ELIMINAR RESERVA ---"

MIENTRAS VERDADERO HACER
    INTENTAR
    LEER id_usuario
    SALIR DEL MIENTRAS
    CAPTURAR ERROR
    MOSTRAR "Debe ingresar un número válido"
    FIN INTENTAR
FIN MIENTRAS

reservas_usuario = FILTRAR reser DONDE id_usuario coincida

SI reservas_usuario está vacía ENTONCES
```

```
        MOSTRAR "No tienes reservas registradas"
        SALIR
FIN SI

MOSTRAR "Tus reservas:"

PARA cada r EN reservas_usuario HACER
        MOSTRAR r.detalles
FIN PARA

MIENTRAS VERDADERO HACER

        LEER id_vuelo

        SI longitud(id_vuelo) >= 4 Y
        primeras 2 letras SON letras Y
        resto SON números ENTONCES
        SALIR DEL MIENTRAS
        SINO
        MOSTRAR "Código inválido"
        FIN SI

FIN MIENTRAS

reserva_encontrada = NONE
```

```
PARA cada r EN reservas_usuario HACER
    SI r.codigo_vuelo = id_vuelo ENTONCES
    reserva_encontrada = r
    SALIR DEL PARA
    FIN SI
FIN PARA

SI reserva_encontrada = NULO ENTONCES
    MOSTRAR "No se encontró esa reserva"
    SALIR
FIN SI

LEER confirmacion

SI confirmacion ≠ "s" ENTONCES
    MOSTRAR "Operación cancelada"
    SALIR
FIN SI

PARA cada v EN vuelos HACER
    SI v.codigo = id_vuelo ENTONCES
    AGREGAR reserva_encontrada.asiento A v.asientos_disponibles
    SALIR DEL PARA
    FIN SI
FIN PARA
```

```
ELIMINAR reserva_encontrada DE reser

MOSTRAR "Reserva eliminada correctamente"

FIN FUNCION




FUNCIÓN eliminar_reserva(lista_reservas, vuelos, id_usuario, id_vuelo)

PARA cada r EN lista_reservas HACER

    SI r.id_usuario = id_usuario Y r.codigo_vuelo = id_vuelo ENTONCES

    PARA cada v EN vuelos HACER
    SI v.codigo = id_vuelo ENTONCES
        AGREGAR r.asiento A v.asientos_disponibles
        SALIR DEL PARA
    FIN SI
    FIN PARA

    ELIMINAR r DE lista_reservas

    RETORNAR VERDADERO

    FIN SI
```

FIN PARA

RETORNAR FALSO

FIN FUNCIÓN

FUNCION conocer_codigo(usu)

MOSTRAR "Ingrese su nombre para conocer su código de usuario:"

LEER nombre_usuario

PARA cada i EN usu HACER

SI nombre_usuario = i.nombre_pasajero ENTONCES

MOSTRAR "Su código de usuario es: " + i.id_usuario + " y el codigo de su vuelo es: " + i.id_vuelo

SALIR DEL PARA

FIN SI

FIN PARA

SI no se encontró ENTONCES

MOSTRAR "Usuario no encontrado. Por favor, ingrese un nombre válido."

FIN SI

FUNCION conocer_codigo(usuarios, reservas)

MOSTRAR "--- CONSULTAR CÓDIGO ---"

```
MIENTRAS VERDADERO HACER
    LEER nombre

    SI nombre contiene solo letras ENTONCES
    SALIR DEL MIENTRAS
    SINO
    MOSTRAR "Nombre inválido"
    FIN SI
FIN MIENTRAS

usuario_encontrado ← NULO

PARA cada u EN usuarios HACER
    SI u.nombre_pasajero = nombre ENTONCES
    usuario_encontrado ← u
    SALIR DEL PARA
    FIN SI
FIN PARA

SI usuario_encontrado = NULO ENTONCES
    MOSTRAR "Usuario no encontrado"
    SALIR
FIN SI

id_usuario = usuario_encontrado.id_usuario
```

```
MOSTRAR "Tu ID de usuario es: " + id_usuario

reservas_usuario = FILTRAR reservas DONDE id_usuario coincida

SI reservas_usuario está vacía ENTONCES
    MOSTRAR "No tienes reservas registradas"
    SALIR
FIN SI

MOSTRAR "Tus vuelos:"

PARA cada r EN reservas_usuario HACER
    MOSTRAR "Vuelo: " + r.codigo_vuelo + " | Asiento: " + r.asiento
FIN PARA

FIN FUNCION
FUNCION conocer_reservas(reser)

MOSTRAR "Ingrese su ID de usuario para conocer sus reservas:"
LEER id_usuario

reservas_usuario = [ ]

PARA cada i EN reser HACER
    SI id_usuario = i.id_usuario ENTONCES
```

```
        MOSTRAR i.detalles
        AGREGAR i A reservas_usuario
        FIN SI
FIN PARA

SI reservas_usuario está vacía ENTONCES
        MOSTRAR "No tienes reservas registradas."
SINO
        MOSTRAR "Tienes " + cantidad(reservas_usuario) + " reservas registradas."
FIN SI


FUNCION conocer_reservas(reser)

MOSTRAR "--- MIS RESERVAS ---"

MIENTRAS VERDADERO HACER
        INTENTAR
        LEER id_usuario
        SALIR DEL MIENTRAS
        CAPTURAR ERROR
        MOSTRAR "Debe ingresar un número válido"
        FIN INTENTAR
FIN MIENTRAS

reservas_usuario = FILTRAR reser DONDE id_usuario coincida
```

```
SI reservas_usuario está vacía ENTONCES
    MOSTRAR "No tienes reservas registradas"
    SALIR
FIN SI

MOSTRAR "Tienes " + cantidad(reservas_usuario) + " reservas"

PARA cada r EN reservas_usuario CON índice idx HACER
    MOSTRAR idx + ". " + r.detalles
FIN PARA


FUNCIÓN pedir_entero(mensaje)

MIENTRAS VERDADERO HACER
    INTENTAR
    LEER valor
    RETORNAR valor
    CAPTURAR ERROR
    MOSTRAR "Ingrese un número válido"
    FIN INTENTAR
FIN MIENTRAS

FIN FUNCIÓN
```

```
FUNCIÓN pedir_opcion(mensaje, opciones)

MIENTRAS VERDADERO HACER
    LEER op

    SI op ESTÁ EN opciones ENTONCES
    RETORNAR op
    SINO
    MOSTRAR "Opción inválida"
    FIN SI
FIN MIENTRAS

FIN FUNCIÓN


FUNCIÓN pedir_asiento()

MIENTRAS VERDADERO HACER
    a = pedir_entero("Número de asiento nuevo (1-35)")

    SI 1 <= a <= 35 ENTONCES
    RETORNAR a
    SINO
    MOSTRAR "Asiento fuera de rango"
    FIN SI
```

```
FIN MIENTRAS

FIN FUNCIÓN



FUNCIÓN buscar_vuelo(vuelos, codigo)

PARA cada v EN vuelos HACER
    SI v.codigo = codigo ENTONCES
    RETORNAR v
    FIN SI
FIN PARA

RETORNAR NONE

FIN FUNCIÓN



FUNCION cambiar_asiento(reservas, vuelos)

id_usuario = pedir_entero("ID usuario")

PARA cada r EN reservas HACER
    SI r.id_usuario = id_usuario ENTONCES

    MOSTRAR r.detalles
```

```
        nuevo = pedir_asiento()

        vuelo = buscar_vuelo(vuelos, r.codigo_vuelo)

        SI NO vuelo.ocupar_asiento(nuevo) ENTONCES
        MOSTRAR "Asiento ocupado"
        SALIR
        FIN SI

        AGREGAR r.asiento A vuelo.asientos_disponibles

        ACTUALIZAR r.asiento = nuevo

        MOSTRAR "Asiento cambiado"
        SALIR

        FIN SI
FIN PARA

MOSTRAR "No encontrado"


FUNCION cambiar_asiento(reservas, vuelos)

MOSTRAR "--- CAMBIO DE ASIENTO ---"
```

```
MIENTRAS VERDADERO HACER
    INTENTAR
    LEER id_usuario
    SALIR DEL MIENTRAS
    CAPTURAR ERROR
    MOSTRAR "Debe ingresar un número válido"
    FIN INTENTAR
FIN MIENTRAS

reservas_usuario = FILTRAR reservas DONDE id_usuario coincida

SI reservas_usuario está vacía ENTONCES
    MOSTRAR "No tienes reservas"
    SALIR
FIN SI

MOSTRAR "Tus reservas:"

PARA cada r EN reservas_usuario CON índice idx HACER
    MOSTRAR idx + ". " + r.detalles
FIN PARA

MIENTRAS VERDADERO HACER
    INTENTAR
    LEER seleccion
```

```
        SI 1 <= seleccion <= cantidad(reservas_usuario) ENTONCES
        reserva = reservas_usuario[seleccion]
        SALIR DEL MIENTRAS
        SINO
        MOSTRAR "Opción fuera de rango"
        FIN SI

        CAPTURAR ERROR
        MOSTRAR "Debe ingresar un número válido"
        FIN INTENTAR
FIN MIENTRAS

vuelo =NONE

PARA cada v EN vuelos HACER
        SI v.codigo = reserva.codigo_vuelo ENTONCES
        vuelo = v
        SALIR DEL PARA
        FIN SI
FIN PARA

SI vuelo = NULO ENTONCES
        MOSTRAR "Error interno: vuelo no encontrado"
        SALIR
FIN SI
```

```
MOSTRAR mapa_asientos(vuelo)

MIENTRAS VERDADERO HACER
    INTENTAR
    LEER nuevo

    SI 1 <= nuevo <= 35 ENTONCES
    SALIR DEL MIENTRAS
    SINO
    MOSTRAR "Fuera de rango"
    FIN SI

    CAPTURAR ERROR
    MOSTRAR "Debe ingresar un número"
    FIN INTENTAR
FIN MIENTRAS

SI NO vuelo.ocupar_asiento(nuevo) ENTONCES
    MOSTRAR "Asiento ocupado"
    SALIR
FIN SI

AGREGAR reserva.asiento A vuelo.asientos_disponibles

ACTUALIZAR reserva.asiento ← nuevo
```

```
MOSTRAR "Asiento cambiado correctamente"

FIN FUNCION
FUNCION ver_vuelos_disponibles(vuelos)

MOSTRAR "--- VUELOS DISPONIBLES ---"

destinos ← {
        "1":"Madrid",
        "2":"Paris",
        "3":"Roma",
        "4":"Londres"
}

MIENTRAS VERDADERO HACER

        MOSTRAR "Seleccione destino:"

        PARA cada k, v EN destinos HACER
        MOSTRAR k + ". " + v
        FIN PARA

        LEER opcion

        SI opcion ESTÁ EN destinos ENTONCES
```

```
        destino ← destinos[opcion]
        SALIR DEL MIENTRAS
        SINO
        MOSTRAR "Opción inválida"
        FIN SI

FIN MIENTRAS

vuelos_filtrados = FILTRAR vuelos DONDE destino coincida

SI vuelos_filtrados está vacío ENTONCES
        MOSTRAR "No hay vuelos disponibles"
        SALIR
FIN SI

MOSTRAR "Vuelos hacia " + destino

PARA cada v EN vuelos_filtrados CON índice idx HACER

        total = 35
        disponibles = cantidad(v.asientos_disponibles)
        ocupados = total - disponibles

        MOSTRAR idx + ". " + v.info_vuelo
        MOSTRAR "Disponibles: " + disponibles + " | Ocupados: " + ocupados
```

```
FIN PARA



FUNCION crear_menu(vuelos, usuarios, reservas)

CREAR ventana

MOSTRAR "Sistema de Aerolínea"

FUNCION reservar()

    CREAR ventana_reserva

    LEER destino
    LEER codigo
    LEER asiento
    LEER nombre
    LEER apellido

    INTENTAR
    SI NO (1 <= asiento <= 35) ENTONCES
    ERROR
    FIN SI
    CAPTURAR ERROR
    MOSTRAR "Asiento inválido"
```

```
SALIR
FIN INTENTAR

vuelo = NONE

PARA cada v EN vuelos HACER
SI v.codigo = codigo Y v.destino = destino ENTONCES
vuelo = v
SALIR DEL PARA
FIN SI
FIN PARA

SI vuelo = NULO ENTONCES
MOSTRAR "Vuelo no encontrado"
SALIR
FIN SI

SI NO vuelo.ocupar_asiento(asiento) ENTONCES
MOSTRAR "Asiento ocupado"
SALIR
FIN SI

id_usu = número aleatorio

CREAR usuario(nombre, apellido, id_usu)
AGREGAR usuario A usuarios
```

```
        SI 1 <= asiento <= 10 ENTONCES
        CREAR reserva_premium
        SINO SI 11 <= asiento <= 20 ENTONCES
        CREAR reserva_normal
        SINO
        CREAR reserva_economica
        FIN SI

        AGREGAR reserva A reservas

        MOSTRAR "Reserva creada con éxito"
        MOSTRAR "ID usuario: " + id_usu

        CERRAR ventana_reserva

FIN FUNCION



MOSTRAR botón "Confirmar" QUE EJECUTA reservar


FIN FUNCION
FUNCION ver_reservas(reservas)


texto = ""
```

```
PARA cada r EN reservas HACER
        texto = texto + r.detalles + salto_linea
FIN PARA

SI texto = "" ENTONCES
        texto = "No hay reservas"
FIN SI

MOSTRAR "Reservas:"
MOSTRAR texto


FUNCION eliminar()

MOSTRAR "Abrir eliminación"


FUNCION cambiar()

MOSTRAR "Abrir cambio de asiento"


FUNCION stats(vuelos)
```

```
texto = ""

PARA cada v EN vuelos HACER

    total = 35
    libres = cantidad(v.asientos_disponibles)
    ocupados = total - libres

    texto = texto + v.codigo + " → " + ocupados + "/" + total + salto_linea

FIN PARA

MOSTRAR "Estadísticas:"
MOSTRAR texto


FUNCION salir(ventana)

CERRAR ventana


FUNCION crear_interfaz(ventana)
```

MOSTRAR "Sistema Aerolínea"

MOSTRAR botón "Reservar" QUE EJECUTA reservar

MOSTRAR botón "Ver Reservas" QUE EJECUTA ver_reservas

MOSTRAR botón "Eliminar Reserva" QUE EJECUTA eliminar

MOSTRAR botón "Cambiar Asiento" QUE EJECUTA cambiar

MOSTRAR botón "Estadísticas" QUE EJECUTA stats

MOSTRAR botón "Salir" QUE EJECUTA salir

INICIAR interfaz

FUNCION guardar_datos(vuelos, usuarios, reservas)

ABRIR archivo "datos_aerolinea.pkl" EN modo escritura

GUARDAR vuelos, usuarios, reservas EN archivo

CERRAR archivo

FUNCIÓN cargar_datos()

SI archivo "datos_aerolinea.pkl" existe ENTONCES

```
        ABRIR archivo EN modo lectura

        CARGAR vuelos, usuarios, reservas

        MOSTRAR "Datos cargados correctamente"

        RETORNAR vuelos, usuarios, reservas

FIN SI

RETORNAR NULO

FIN FUNCIÓN
FUNCION main()

datos = cargar_datos()

SI datos EXISTE ENTONCES
        vuelos_disponibles, usuarios_registrados, reservas_realizadas ← datos
SINO

        MOSTRAR "Primera ejecución: creando datos iniciales"

        vuelos_disponibles = [ ]
        usuarios_registrados = [ ]
```

reservas_realizadas = [ ]

CREAR vuelo1 ("Madrid", 35, "AV123", 500, fecha)

CREAR vuelo11 ("Madrid", 35, "AV124", 500, fecha)

CREAR vuelo12 ("Madrid", 35, "AV125", 500, fecha)

AGREGAR vuelo1, vuelo11, vuelo12 A vuelos_disponibles

OCUPAR asiento 3 EN vuelo1

OCUPAR asiento 7 EN vuelo1

OCUPAR asiento 15 EN vuelo1

CREAR vuelo2 ("Paris", 35, "AV456", 450, fecha)

CREAR vuelo21 ("Paris", 35, "AV457", 450, fecha)

CREAR vuelo22 ("Paris", 35, "AV458", 450, fecha)

AGREGAR vuelo2, vuelo21, vuelo22 A vuelos_disponibles

CREAR vuelo3 ("Roma", 35, "AV789", 400, fecha)

CREAR vuelo31 ("Roma", 35, "AV790", 400, fecha)

CREAR vuelo32 ("Roma", 35, "AV791", 400, fecha)

AGREGAR vuelo3, vuelo31, vuelo32 A vuelos_disponibles

CREAR vuelo4 ("Londres", 35, "AV321", 550, fecha)

CREAR vuelo41 ("Londres", 35, "AV322", 550, fecha)

CREAR vuelo42 ("Londres", 35, "AV323", 550, fecha)

AGREGAR vuelo4, vuelo41, vuelo42 A vuelos_disponibles

CREAR usuario1 ("Juan", "Perez", 1111)

CREAR usuario2 ("Ana", "Lopez", 2222)

AGREGAR usuario1, usuario2 A usuarios_registrados

OCUPAR asiento 5 EN vuelo1

CREAR reserva1 (Premium, Juan, vuelo AV123, asiento 5)

OCUPAR asiento 6 EN vuelo1

CREAR reserva2 (Normal, Ana, vuelo AV123, asiento 6)

OCUPAR asiento 3 EN vuelo2

CREAR reserva3 (Economica, Juan, vuelo AV456, asiento 3)

AGREGAR reserva1, reserva2, reserva3 A reservas_realizadas

FIN SI

```
MOSTRAR "1. Consola / 2. Gráfico"

LEER modo


SI modo = "1" ENTONCES


        LLAMAR crear_menu(vuelos_disponibles, usuarios_registrados,
reservas_realizadas)


SINO


        MIENTRAS VERDADERO HACER


        MOSTRAR "====== AEROLÍNEA ======"

        MOSTRAR "1. Hacer reserva"

        MOSTRAR "2. Conocer mi código"

        MOSTRAR "3. Eliminar reserva"

        MOSTRAR "4. Ver mis reservas"

        MOSTRAR "5. Ver vuelos"

        MOSTRAR "6. Cambiar asiento"

        MOSTRAR "7. Salir"


        LEER opcion


        SI opcion NO ESTÁ EN [1..7] ENTONCES

        MOSTRAR "Opción inválida"
```

```
CONTINUAR
FIN SI

SEGÚN opcion HACER

CASO "1":
        LLAMAR hacer_reserva

CASO "2":
        LLAMAR conocer_codigo

CASO "3":
        LLAMAR eli_reserva

CASO "4":
        LLAMAR conocer_reservas

CASO "5":
        LLAMAR ver_vuelos_disponibles

CASO "6":
        LLAMAR cambiar_asiento

CASO "7":
        LLAMAR guardar_datos
        MOSTRAR "Datos guardados correctamente"
```

MOSTRAR FIN

- Diagramas de flujo:

App:https://drive.google.com/file/d/1yGqvcMV2NRTMgSaCdZtb93a6UHaF3fSa/view?usp=drive_link

Clases:https://uao-my.sharepoint.com/:w:/g/personal/juan_garciamejia_uao_edu_co/IQByhyOo0nGOSrTNMnWVzCSRAXGhZRw5rhDoLj_0BvaVh0k?e=wSEvDJ